Event Registration & Ticket Payment Platform: SRS Addendum v1.1

Integration Contract Pack · Web Development, Group 5 · Supplements SRS v1.0 (8 October 2026) · Draft for the Day 2 freeze

0. Purpose and precedence

SRS v1.0 defines what the system does. This addendum adds what each workstream needs to build alone and still merge cleanly: decisions that fix v1.0 inconsistencies, shared conventions, the full schema, exact API contracts with JSON examples, shared PHP function signatures, a page map, and an integration plan. Where this addendum and v1.0 disagree, this addendum wins. Values marked PLACEHOLDER must be confirmed on Day 1.

1. Decisions that fix v1.0

ID	Topic	Decision
D1	Failed payments	Every payment attempt is recorded. A failed attempt inserts a payments row (status failed) outside the ticket-issuing transaction, and the order stays pending so the buyer can retry. The order status failed is removed from the schema.
D2	Quota	FR17 becomes Must. Quota is checked softly at order creation and hard (row lock) at payment.
D3	Categories	Four active categories: Student, Staff, General Public, VIP. attendee_type is informational only and does not restrict which category a user can buy.
D4	QR library	Bundled locally at /public/js/vendor/qrcode.min.js. No CDN.
D5	Verify by number	verify-ticket.php takes one field, value, which is either the QR code or a typed ticket number. One atomic UPDATE matches either.

ID	Topic	Decision
D6	Must-have list	The must-haves are the M rows of Section 5 in v1.0 (R4 should point there, not to Section 12).
D7	Verification log	FR18 stays Should-have, but the table exists, so verify-ticket.php writes one row per attempt.
D8	Deadline	Day numbers count from kickoff. Confirm the real deadline on Day 1. If it is shorter than 14 days, keep the order of the checkpoints in Section 11, shrink the gaps, and drop stretch goals.
D9	Status codes	Add 402 (payment failed) and 405 (wrong HTTP method) to the v1.0 list.

2. Shared conventions

- **Folder and URLs:** the repo folder must be named uiconf inside the web root (htdocs or Laragon www). Pages and API are both served by Apache on the same origin, so there are no CORS or cookie problems. Do not open pages that call the API through the VS Code live-server extension. Pages: http://localhost/uiconf/public/ and API: http://localhost/uiconf/api/.
- **Base path:** defined once in public/js/config.js as API_BASE = "/uiconf/api/".
- **Requests:** JSON bodies with Content-Type: application/json; PHP reads php://input. fetch always uses credentials: "same-origin".
- **Naming:** snake_case for DB columns and JSON keys. Endpoint files use hyphens (verify-ticket.php, update-price.php).
- **Money:** DECIMAL(10,2) in the database; sent as JSON strings with two decimals ("2000.00"). Currency is NGN, shown as ₦.
- **Dates:** DATETIME in Africa/Lagos time, sent as "YYYY-MM-DD HH:MM:SS". db.php sets both the PHP and MySQL time zone.
- **Sessions:** session_start() is called only inside auth.php. Session keys: user_id, full_name, role. Regenerate the session ID on login. Cookie flags HttpOnly and SameSite=Lax.
- **Config:** includes/config.example.php is committed; includes/config.php (real credentials) is in .gitignore. Constants: DB_HOST, DB_NAME (uiconf), DB_USER, DB_PASS, PAYMENT_MODE ('simulated'), MAX_TICKETS_PER_ORDER (10),

TICKET_PREFIX ('UIT26').

Response envelope (every endpoint):

```
{"success": true, "message": "Order created", "data": {}, "errors": []}
```

Validation failure (HTTP 400):

```
{"success": false, "message": "Please fix the highlighted fields", "data":
{}, "errors": [{"field": "email", "message": "Enter a valid email address"}]}
```

Status codes: 200, 201, 400, 401 (not logged in), 402 (payment failed), 403 (wrong role), 404, 405, 409 (duplicate or sold out), 500. Never expose raw database errors; log them and return a friendly message.

3. Database: database/schema.sql (owner BE-A)

```
CREATE DATABASE IF NOT EXISTS uiconf CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE uiconf;

CREATE TABLE events (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  event_date DATETIME NOT NULL,
  venue VARCHAR(200) NOT NULL,
  description TEXT NOT NULL,
  schedule TEXT NOT NULL,
  speakers TEXT NOT NULL,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE users (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  phone VARCHAR(20) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  attendee_type ENUM('student','staff','public','vip') NOT NULL DEFAULT
'public',
  role ENUM('attendee','admin','verifier') NOT NULL DEFAULT 'attendee',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
) ENGINE=InnoDB;

CREATE TABLE ticket_categories (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  event_id INT UNSIGNED NOT NULL,
  name VARCHAR(50) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  quota INT UNSIGNED NOT NULL,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  FOREIGN KEY (event_id) REFERENCES events(id)
) ENGINE=InnoDB;

CREATE TABLE orders (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id INT UNSIGNED NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  status ENUM('pending','paid','cancelled') NOT NULL DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
) ENGINE=InnoDB;

CREATE TABLE order_items (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_id INT UNSIGNED NOT NULL,
  category_id INT UNSIGNED NOT NULL,
  quantity INT UNSIGNED NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  line_total DECIMAL(10,2) NOT NULL,
  UNIQUE KEY uq_order_category (order_id, category_id),
  FOREIGN KEY (order_id) REFERENCES orders(id),
  FOREIGN KEY (category_id) REFERENCES ticket_categories(id)
) ENGINE=InnoDB;

CREATE TABLE payments (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_id INT UNSIGNED NOT NULL,
  reference VARCHAR(40) NOT NULL UNIQUE,
  amount DECIMAL(10,2) NOT NULL,
  method VARCHAR(20) NOT NULL DEFAULT 'simulated',
  status ENUM('success','failed') NOT NULL,
  provider_response TEXT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (order_id) REFERENCES orders(id)
```

```
) ENGINE=InnoDB;

CREATE TABLE tickets (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_id INT UNSIGNED NOT NULL,
  category_id INT UNSIGNED NOT NULL,
  ticket_number VARCHAR(20) NOT NULL UNIQUE,
  verification_code CHAR(32) NOT NULL UNIQUE,
  status ENUM('issued','used') NOT NULL DEFAULT 'issued',
  issued_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  used_at DATETIME NULL,
  verified_by INT UNSIGNED NULL,
  FOREIGN KEY (order_id) REFERENCES orders(id),
  FOREIGN KEY (category_id) REFERENCES ticket_categories(id),
  FOREIGN KEY (verified_by) REFERENCES users(id)
) ENGINE=InnoDB;

CREATE TABLE verification_log (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  ticket_id INT UNSIGNED NULL,
  input_code VARCHAR(100) NOT NULL,
  result ENUM('valid','used','invalid') NOT NULL,
  officer_id INT UNSIGNED NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (ticket_id) REFERENCES tickets(id),
  FOREIGN KEY (officer_id) REFERENCES users(id)
) ENGINE=InnoDB;
```

Relationships: one user has many orders; one order has many items, payments and tickets; one category has many tickets. unit_price is copied into order_items so later price edits never change old orders.

4. Seed data: database/seed.sql

Event details, prices and quotas below are PLACEHOLDERS to confirm on Day 1. The hash is the widely used bcrypt hash of the word password, for local development only. If login fails, generate a fresh one with php -r 'echo password_hash("password", PASSWORD_DEFAULT);' and replace it.

```
USE uiconf;
SET @h = '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi';

INSERT INTO events (title, event_date, venue, description, schedule,
speakers) VALUES
('University of Ibadan Technology Conference 2026', '2026-11-20 09:00:00',
'University of Ibadan (PLACEHOLDER)',
 'PLACEHOLDER description', '09:00 Registration | 10:00 Keynote
(PLACEHOLDER)', 'PLACEHOLDER speakers');

INSERT INTO ticket_categories (event_id, name, price, quota) VALUES
(1, 'Student', 2000.00, 200), (1, 'Staff', 5000.00, 100),
(1, 'General Public', 7500.00, 150), (1, 'VIP', 20000.00, 50);

INSERT INTO users (full_name, email, phone, password_hash, attendee_type,
role) VALUES
('Admin User', 'admin@uiconf.test', '08000000001', @h, 'staff', 'admin'),
('Gate Officer', 'verifier@uiconf.test', '08000000002', @h, 'staff',
'verifier'),
('Test Attendee', 'attendee@uiconf.test', '08000000003', @h, 'student',
'attendee');
```

5. Validation rules (all enforced server-side)

Field	Rule
full_name	2 to 100 characters, trimmed
email	valid format, at most 150 characters, stored lowercase, unique
phone	10 to 15 digits, optional leading +; spaces and dashes removed
password	8 to 72 characters (72 is the bcrypt limit)
attendee_type	student, staff, public or vip
quantity	whole number 1 to 10 per line; total across the order at most MAX_TICKETS_PER_ORDER; each category appears once per order

Field	Rule
price	0.00 to 1000000.00, two decimals
reference	PAY- followed by 16 to 32 hex characters, generated by the browser once per payment attempt
value (verify)	1 to 100 characters, trimmed
q (search)	up to 50 characters, bound as a prepared LIKE parameter
page	integer 1 or more; per_page is fixed at 20

Ticket number: TICKET_PREFIX, a hyphen, then 5 characters from ABCDEFGHJKLMNPQRSTUVWXYZ23456789 chosen with random_int (no 0, O, 1 or I). Retry up to 5 times on a duplicate-key error. Verification code: bin2hex(random_bytes(16)), 32 hex characters.

6. API contract

Every response uses the envelope from Section 2; the examples show data only. Protected endpoints return 401 when logged out and 403 for the wrong role. Wrong HTTP method returns 405. Admin endpoints are under /api/admin/.

6.1 Public and account endpoints

```
GET /api/event.php   (public, BE-B)
200 data: {"event": {"id": 1, "title": "University of Ibadan Technology
Conference 2026", "event_date": "2026-11-20 09:00:00", "venue": "University
of Ibadan", "description": "...", "schedule": "...", "speakers": "..."}}

GET /api/categories.php   (public, BE-B, active categories only)
200 data: {"categories": [{"id": 1, "name": "Student", "price": "2000.00",
"quota": 200, "remaining": 173, "sold_out": false}]}

POST /api/register.php   (public, BE-A)
request: {"full_name": "Ada Obi", "email": "ada@example.com", "phone":
"08012345678", "password": "Secret123", "attendee_type": "student"}
201 data: {"user_id": 12}   (no automatic login; the page sends the user to
login.html)
400 errors per field   409 errors: [{"field": "email", "message": "This email
```

```
is already registered"}]

POST /api/login.php   (public, BE-A)
request: {"email": "ada@example.com", "password": "Secret123"}
200 data: {"user": {"id": 12, "full_name": "Ada Obi", "role": "attendee"}}
401 message: "Incorrect email or password" (same text for unknown email and
wrong password)

POST /api/logout.php   (user, BE-A)
200 data: {}

GET /api/me.php   (public, BE-A)
200 data: {"user": {"id": 12, "full_name": "Ada Obi", "role": "attendee"}}
or {"user": null} when logged out (still 200, so the nav bar never treats it
as an error)
```

6.2 Orders, payment and tickets (TP-A and TP-B)

```
POST /api/order.php   (user, TP-A)
request: {"items": [{"category_id": 1, "quantity": 2}, {"category_id": 4,
"quantity": 1}]}
201 data: {"order": {"id": 45, "status": "pending", "total_amount":
"24000.00", "lines": [
  {"category_id": 1, "category_name": "Student", "quantity": 2, "unit_price":
"2000.00", "line_total": "4000.00"},
  {"category_id": 4, "category_name": "VIP", "quantity": 1, "unit_price":
"20000.00", "line_total": "20000.00"}]}}
400 bad quantity, repeated category or over the order cap   404 unknown
category
409 inactive or sold out, e.g. message "VIP is sold out"

GET /api/order.php?id=45   (user, owner only, TP-A; used to resume checkout)
200 data: same "order" object as above   404 if not found or not yours

POST /api/payment.php   (user, TP-A)
request: {"order_id": 45, "reference": "PAY-3F9A1C7E5B2D4A68", "method":
"simulated", "simulate_outcome": "success"}
  simulate_outcome (success or fail) is read only when PAYMENT_MODE is
simulated. No card data is ever stored.
200 data: {"payment": {"reference": "PAY-3F9A1C7E5B2D4A68", "status":
"success", "amount": "24000.00", "paid_at": "2026-11-02 14:05:31"}, "order":
{"id": 45, "status": "paid"}, "ticket_count": 3}
```

```
402 data: {"payment": {"reference": "PAY-...", "status": "failed", "amount":
"24000.00"}, "order": {"id": 45, "status": "pending"}}   message "Payment
failed. You can try again."
409 order already paid or closed, or sold out at payment time
Repeated reference: returns the stored result again (200 or 402) and creates
nothing new.

GET /api/ticket.php?order_id=45   (user, owner only, TP-B)
200 data: {"tickets": [{"id": 101, "ticket_number": "UIT26-7K4Q9",
"verification_code": "9f2c4a1e7b3d4c58a6f0e1d2c3b4a596", "category":
"Student", "status": "issued", "holder_name": "Ada Obi", "event_title":
"University of Ibadan Technology Conference 2026", "event_date": "2026-11-20
09:00:00"}]}
404 if the order is not yours or not paid

GET /api/my-tickets.php   (user, TP-B, newest first)
200 data: {"tickets": [ same fields as above plus "order_id": 45 and
"issued_at": "2026-11-02 14:05:31" ]}

POST /api/verify-ticket.php   (verifier or admin, TP-B)
request: {"value": "UIT26-7K4Q9"}   (QR code text or typed ticket number)
All three outcomes return HTTP 200 with success true; the page switches on
data.result.
200 data: {"result": "valid", "ticket": {"ticket_number": "UIT26-7K4Q9",
"category": "Student", "holder_name": "Ada Obi", "used_at": "2026-11-20
10:42:07"}}   message "Valid ticket. Admit."
200 data: {"result": "used", "ticket": {... "used_at": "<time of the first
scan>"}}   message "Already used"
200 data: {"result": "invalid", "ticket": null}   message "Ticket not found"
```

6.3 Admin endpoints (BE-B)

```
GET /api/admin/stats.php
200 data: {"totals": {"orders_paid": 38, "tickets_sold": 91, "tickets_used":
57, "revenue": "412000.00"},
  "per_category": [{"category_id": 1, "name": "Student", "price": "2000.00",
"quota": 200, "sold": 60, "used": 40, "revenue": "120000.00"}]}
  Revenue counts paid orders only.

GET /api/admin/attendees.php?q=ada&page=1   (one row per ticket)
200 data: {"attendees": [{"ticket_number": "UIT26-7K4Q9", "holder_name": "Ada
Obi", "email": "ada@example.com", "phone": "08012345678", "category":
```

```
"Student", "status": "issued", "issued_at": "...", "used_at": null}], "page":
1, "per_page": 20, "total": 91}
  q matches name, email or ticket number.

GET /api/admin/orders.php?page=1
200 data: {"orders": [{"id": 45, "user_name": "Ada Obi", "email":
"ada@example.com", "total_amount": "24000.00", "status": "paid",
"created_at": "...", "payments": [{"reference": "PAY-...", "status":
"success", "amount": "24000.00", "created_at": "..."}]}], "page": 1,
"per_page": 20, "total": 38}

GET /api/admin/users.php?page=1
200 data: {"users": [{"id": 12, "full_name": "Ada Obi", "email":
"ada@example.com", "phone": "08012345678", "attendee_type": "student",
"role": "attendee", "created_at": "..."}], "page": 1, "per_page": 20,
"total": 52}

POST /api/admin/update-price.php
request: {"category_id": 1, "price": "2500.00"}
200 data: {"category": {"id": 1, "name": "Student", "price": "2500.00"}}
400 invalid price   404 unknown category

POST /api/admin/update-event.php   (all six fields required)
request: {"title": "...", "event_date": "2026-11-20 09:00:00", "venue":
"...", "description": "...", "schedule": "...", "speakers": "..."}
200 data: {"event": { ...updated event, same shape as event.php... }}
```

7. Shared PHP interfaces (signatures are frozen; internals are the owner's choice)

Every endpoint follows the same order: require_method, then require_login or require_role, then read_json and validate, then prepared queries, then ok() or fail().

```
// includes/db.php (BE-A)
function db(): PDO;   // singleton; ERRMODE_EXCEPTION, FETCH_ASSOC,
EMULATE_PREPARES=false, utf8mb4, Africa/Lagos

// includes/response.php (BE-B)
function json_response(bool $success, string $message, array $data = [],
array $errors = [], int $status = 200): never;
function ok(string $message, array $data = [], int $status = 200): never;
```

```
function fail(string $message, int $status, array $errors = [], array $data =
[]): never;
function read_json(): array;   // decodes php://input; 400 on malformed JSON
// plus a global exception handler: log the error, then fail('Something went
wrong', 500)

// includes/validate.php (BE-B)
function require_method(string $method): void;   // 405 otherwise
function clean_string(mixed $v, int $min, int $max): ?string;   // null when
invalid
function valid_email(mixed $v): ?string;
function valid_int(mixed $v, int $min, int $max): ?int;
function valid_money(mixed $v, string $min, string $max): ?string;

// includes/auth.php (BE-A)
function current_user(): ?array;   // ['id' => int, 'full_name' => string,
'role' => string] or null
function require_login(): array;   // 401 if not logged in
function require_role(string ...$roles): array;   // 401 or 403; verify-
ticket uses require_role('verifier', 'admin')
function login_user(array $userRow): void;   // regenerates session id, sets
session keys
function logout_user(): void;

// includes/pricing.php (TP-A)
function build_order_lines(PDO $pdo, array $items): array;   // checks items
against the DB; returns ['lines' => [...], 'total' => '24000.00']; calls
fail() on bad input

// includes/payment_gateway.php (TP-A): the only file that changes for a real
gateway
function process_payment(array $order, array $input): array;   // ['status'
=> 'success'|'failed', 'provider_response' => string]

// includes/ticketing.php (TP-B)
function generate_ticket_number(PDO $pdo): string;
function generate_verification_code(): string;
function issue_tickets(PDO $pdo, int $orderId): array;   // call ONLY inside
the caller's open transaction; never begins or commits; inserts one ticket
per unit in order_items; returns the new ticket rows
```

On Day 2, BE-A and BE-B push these files with the signatures above, even if the bodies are minimal, so TP and FE can code against them immediately.

8. Payment and verification logic (replaces v1.0 Section 8.3)

payment.php, after auth and validation:

1. If a payments row with this reference exists, return its stored result (200 for success, 402 for failed) and stop.
2. Load the order; it must belong to the user. If status is not pending, return 409.
3. Soft quota check (no lock). If any category lacks enough remaining tickets, return 409 before processing payment.
4. Call process_payment().
5. If it failed: INSERT a payments row (status failed) and return 402. The order stays pending.
6. If it succeeded: BEGIN. Lock the order row and re-check pending. Lock the category rows (SELECT ... FOR UPDATE) and count issued and used tickets per category against quota; if short, ROLLBACK and return 409. INSERT the payments row (status success). UPDATE the order to paid. Call issue_tickets(). COMMIT.
7. If anything throws: ROLLBACK, INSERT a payments row (status failed, provider_response noting the internal error) outside the transaction, log the error, and return 500 with a friendly message.

With a real gateway, a rollback after a successful charge would need a refund. That is out of scope for the simulated prototype and should be noted in the final report.

verify-ticket.php:

```
UPDATE tickets SET status = 'used', used_at = NOW(), verified_by = :officer
WHERE (verification_code = :v1 OR ticket_number = :v2) AND status = 'issued';
```

Bind the same value to :v1 and :v2. With EMULATE_PREPARES off, PDO cannot reuse one named placeholder twice. If exactly one row changed, SELECT the ticket and return valid. If none changed, SELECT by code or number: found means used, not found means invalid. Insert a verification_log row in all three cases.

9. Frontend: page map and api.js contract

Page (public/)	Purpose	Auth	Calls	Owner
index.html	Event info and login prompt	none	event.php, me.php	FE-A

Page (public/)	Purpose	Auth	Calls	Owner
register.html	Registration form	none	register.php	FE-A
login.html	Login; honours ?next= after success	none	login.php	FE-A
tickets.html	Select category and quantity, estimated total	user	categories.php, order.php	FE-A
checkout.html?order_id=45	Order summary and payment; retry on failure	user	order.php (GET), payment.php	FE-A
ticket.html?order_id=45	Payment confirmation banner, QR tickets, print view	user	ticket.php	FE-B
my-tickets.html	All of the user's tickets	user	my-tickets.php	FE-B
verify.html	Gate verification, large tap targets	verifier, admin	verify-ticket.php	FE-B
admin.html	Tabs: stats, attendees, orders, users, price, event	admin	/api/admin/*	FE-B

Purchase path (4 steps): index.html, tickets.html, checkout.html, ticket.html. After a successful payment, checkout.html redirects to ticket.html?order_id=45&paid=1.

Shared files (FE-B owns, FE-A consumes): public/js/config.js (API_BASE, USE_STUBS), public/js/api.js, public/js/nav.js (header driven by me.php), public/css/styles.css. Page scripts live in public/js/pages/.

api.js contract: apiGet(path, params) and apiPost(path, body) always resolve (never throw) with {ok, status, message, data, errors}. A network failure resolves as {ok: false, status: 0, message: "Network error. Check your connection.", data: {}, errors: []}. Any 401 redirects to login.html?next=<current page>.

Stubs: copy the example responses in Section 6 into public/js/stubs/<endpoint>.json. When USE_STUBS is true in config.js, api.js serves them, so frontend work never waits for the backend.

10. Ownership of the seams

Seam	Provider	Consumer
schema.sql, seed.sql, ERD image	BE-A	everyone
db.php, auth.php	BE-A	all PHP
response.php, validate.php	BE-B	all PHP
pricing.php, payment_gateway.php	TP-A	order.php, payment.php
issue_tickets() in ticketing.php	TP-B	payment.php (TP-A)
This contract and the stub JSON	PM	FE-A, FE-B, QA
config.js, api.js, nav.js, styles	FE-B	FE-A and all pages

11. Integration plan

Day	Checkpoint
1	Team confirms D1 to D9 and the PLACEHOLDER values. Repo created with the folder structure, .gitignore and config.example.php.
2	Freeze: schema.sql, seed.sql, this contract, helper stubs pushed, stub JSON files, test plan drafted.
3	db, auth, response and validate merged to develop. Everyone imports schema.sql then seed.sql.
5 to 6	Slice 1: register, login, categories, order, and a stubbed payment, end to end.
8	Slice 2: payment transaction, ticket issuing and the ticket page, end to end.
9	Slice 3: verification and the admin dashboard.
10 to 11	System testing and bug fixing.
12	Feature freeze and documentation review.

Day	Checkpoint
13 to 14	Demo preparation and submission.

After the Day 2 freeze, any change to the schema or contract goes through a pull request that edits this document, approved by the PM and the affected owners, and is announced in the group chat.

Branching: feature branches off develop; one reviewer per pull request; one named integrator (suggested: the PM) merges into develop; main is updated only after QA signs off a slice.

12. Added tests and open items

ID	Scenario	Expected result
T28	Failed simulated payment, then retry with a new reference	Failed payments row, order still pending, no tickets; retry succeeds
T29	Order exceeds remaining quota, or stock vanishes before payment	409 at order creation or at payment; no tickets issued
T30	Verify by typed ticket number	Same valid, used and invalid outcomes as scanning
T31	Any endpoint returns 401	api.js redirects to login.html?next=... and returns after login

To confirm on Day 1: (1) the real deadline; (2) the PLACEHOLDER event date, venue, prices and quotas; (3) simulated or real payment; (4) who the integrator is.